

Proyecto # 3 – Programación Concurrente

Almacén Robótico Concurrente

En vista del éxito del sistema de clasificación lógico, el almacén robótico ha decidido escalar sus operaciones. Ahora, en lugar de un solo robot, la matriz lógica de 6x6 es operada simultáneamente por múltiples robots de carga que deben coordinarse para no colisionar, mientras nuevos cargamentos llegan continuamente. Se requiere implementar una solución concurrente para este problema, utilizando **monitores e hilos (Threads) en Java**.

Se debe tener en consideración lo siguiente:

1. **El Tablero (Recurso Compartido):** el almacén sigue siendo una matriz de 6x6 (coordenadas de la (0,0) a la (5,5)). Esta matriz es el recurso crítico principal. La zona de extracción (la meta) sigue siendo la coordenada (5,5).
2. **Robots (Consumidores):** múltiples robots operan en el tablero. Cada robot (hilo) está definido por su identificador y su "nivel de batería", el cual representa la cantidad de tareas (movimientos o empujes de cajas) que puede realizar antes de apagarse. Su objetivo es encontrar "Cajas Objetivo" y empujarlas hacia la zona de extracción en (5,5).
3. **Cintas Transportadoras (Productores):** son los procesos (hilos) responsables de introducir aleatoriamente nuevas entidades (Cajas Objetivo o Cajas de Bloqueo) en las casillas vacías del tablero.

Archivo de Entrada

La interacción inicial debe establecerse a través de datos de entrada. Al ejecutar el programa debe suministrarse un archivo en formato TXT (el nombre del mismo se recibirá como argumento en el método *main*), ejecutar su lectura en batch y disparar los hilos correspondientes.

El archivo debe seguir el siguiente formato:

```
Cajas_Objetivo_Iniciales, 3
Cajas_Bloqueo_Iniciales, 5
Robots, 4, 15
Productores, 2
```

- Las primeras dos líneas indican cuántas cajas de cada tipo se deben generar aleatoriamente en el tablero al iniciar la simulación.
- La tercera línea indica el número de robots y la batería inicial (cantidad de movimientos) que tiene cada uno.
- La última línea especifica la cantidad de cintas transportadoras (productores) activas.

Requerimientos de Implementación

Se espera que ud. implemente lo siguiente:

1. **Cinta Transportadora (Productor):** la simulación debe incluir la implementación de procesos "productores", responsables de agregar nuevas cajas al tablero siempre que haya al menos una casilla vacía. Si el tablero está completamente lleno, los productores deben entrar en estado de espera. Si un productor intenta colocar una caja y el tablero se llena (saturación), el evento debe ser registrado y reportado al finalizar la ejecución.
2. **Robot de Carga (Consumidor):** los robots deben buscar una Caja Objetivo y empujarla hacia (5,5) respetando las reglas físicas del almacén (no solaparse con otros robots, no empujar dos cajas a la vez). Si no hay Cajas Objetivo en el tablero, el robot debe esperar hasta que un productor ingrese una nueva. Cuando a un robot se le agota la batería (sus tareas permitidas), debe retirarse del tablero y finalizar su hilo. Si aún le queda batería, podrá seguir procesando cajas.

Formato de Salida

Para garantizar la correcta evaluación y trazabilidad de la concurrencia, el programa debe generar una salida estándar por consola estrictamente dividida en dos partes: una **Traza de Ejecución** en tiempo real y un **Reporte Final**. No se permite el uso de interfaces gráficas.

1. **Traza de Ejecución (Log de Eventos):** cada vez que un hilo interactúe con el tablero (recurso compartido) o cambie de estado, debe imprimir una línea en consola siguiendo exactamente esta sintaxis: [Tick/Tiempo] [Identificador_del_Hilo] [ACCIÓN] [Detalles_de_Coordenadas]

Ejemplos de trazas esperadas:

- [Tick-01] [Productor-1] INSERTAR_OBJETIVO -> (1,2)
- [Tick-02] [Productor-2] INSERTAR_BLOQUEO -> (3,3)
- [Tick-03] [Robot-1] MOVER (0,0) -> (0,1) | Batería: 14
- [Tick-04] [Robot-2] EMPUJAR_OBJETIVO (2,2) -> (2,3) | Batería: 9
- [Tick-05] [Productor-1] ESPERA_SATURACION (*Intento de insertar en tablero lleno*)
- [Tick-06] [Robot-1] EXTRACCION_EXITOSA -> (5,5)
- [Tick-07] [Robot-2] BATERIA_AGOTADA -> Retiro del tablero

2. **Reporte Final y Estado del Tablero:** una vez que todos los hilos terminen su ejecución (ya sea por batería agotada en todos los robots o por otra condición de parada que usted defina), el hilo principal (*main*) debe imprimir un resumen estadístico y la matriz resultante:

```
=== REPORTE FINAL DEL ALMACÉN ===
- Cajas Objetivo extraídas (llevadas a 5,5): [Cantidad]
- Saturaciones del tablero (intentos fallidos de productores): [Cantidad]
- Robots que finalizaron por batería agotada: [Cantidad]

=== ESTADO FINAL DEL TABLERO ===
. . X 0 . .
. R . X . .
. . . . .
X . R 0 . .
. . . X . .
. . . . .
```

(Nomenclatura obligatoria para la matriz: R = Robot, O = Caja Objetivo, X = Caja de Bloqueo, . = Casilla Vacía).

Aclaratorias Técnicas y Reglas de Negocio

- **Posición Inicial de las Entidades:** al iniciar la simulación, tanto las cajas iniciales como los robots deben instanciarse y ubicarse en coordenadas aleatorias que se encuentren vacías. Si la configuración del archivo de entrada exige generar un número total de entidades que supera la capacidad de la matriz, el programa debe reportar un error de configuración inicial y finalizar su ejecución.
- **Consumo de Batería:** el estado de espera (cuando un hilo se bloquea aguardando por un recurso o una nueva caja) no consume batería. La batería de un robot solo decrece cuando este realiza un movimiento hacia una casilla vacía o cuando empuja exitosamente una caja.
- **Política de los Productores:** para evitar que un hilo llene el tablero en milisegundos, los productores deben incluir un tiempo de espera o retardo (por ejemplo, mediante `Thread.sleep()`) entre cada intento de inserción. La política exacta de inserción (frecuencia, retardo y probabilidad de generar una Caja Objetivo frente a una de Bloqueo) queda a su criterio, pero **debe estar explícitamente documentada y justificada**.
- **Definición Exacta de "Saturación":** se considera que ocurre una "saturación" únicamente cuando el hilo de un productor se activa para colocar una caja, verifica el estado del monitor (tablero compartido) y confirma que no hay casillas vacías disponibles. En ese instante, debe registrar el intento fallido en la consola y volver a su estado de espera.

Consideraciones para la entrega

- Realice las validaciones adicionales que considere necesarias.
- Ud. debe identificar los procesos, recursos críticos (junto con las operaciones de acceso) y las condiciones de sincronización presentes en el problema para garantizar la exclusión mutua y evitar colisiones (dos robots en la misma celda). Esto lo debe documentar en un archivo en formato **markdown** (README.md).
- Debe diseñar su solución para minimizar o gestionar los interbloqueos.
- Documente también las decisiones de diseño que tome para solucionar el problema e indique las instrucciones de uso.
- No está permitido utilizar soluciones de terceros (esto incluye soluciones generadas por LLMs). Cualquier material consultado o prompts para ideas de solución o documentación debe ser referenciado en una sección del **README.md**, indicando para qué sirvió.
- El proyecto puede realizarse en parejas y la fecha de entrega será el lunes **28/09/2026**, hasta las 11:59 (VET).
- El proyecto se debe entregar en un archivo comprimido cuyo nombre debe seguir la siguiente sintaxis: `<Cedula1>-<Apellido1>-<Cedula2>-<Apellido2>-Proyecto_3.[rar|zip]`. El archivo ZIP/RAR debe contener una carpeta con el mismo nombre del archivo y debe incluir: el archivo README.md, los casos de prueba, el proyecto de código fuente con la implementación de la solución y un archivo makefile o similar para compilar la aplicación **en cualquier plataforma**.
- La entrega debe compilar en una consola/terminal y NO tener dependencias del IDE utilizado.
- El proyecto se debe enviar por e-mail a ldpucv@gmail.com colocando como asunto el nombre del archivo comprimido.

José Yvimas, Septiembre 2026